Master Technical Q&A for Conference Defence

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Debopam Sinha

Defence rule: State exactly what the paper demonstrates. Do not claim broader generalization than the experiment supports. If a limitation is raised, acknowledge it clearly and state the next experiment needed.

1. Hardware Security Fundamentals

Q: What is a Hardware Trojan?
Answer: A Hardware Trojan is an intentional malicious modification of a hardware design. It can remain inactive during normal operation and activate under a rare internal state or input condition.

Q: Why are Hardware Trojans dangerous?
Answer: They can alter functionality, leak sensitive information, reduce reliability, create a backdoor, or cause denial of service. They are dangerous because they may remain dormant during normal verification.

Q: What is RTL?
Answer: RTL stands for Register Transfer Level. It describes digital hardware behavior in terms of registers, data flow, conditional logic, and clocked operations, commonly using Verilog or VHDL.

Q: Why perform Trojan detection at RTL level?
Answer: RTL-level analysis occurs early in the design flow, before synthesis and physical implementation. Early detection can reduce redesign cost and supports security review before fabrication.

Q: What is the difference between RTL, gate-level netlist, and layout?
Answer: RTL describes intended logic behavior. A gate-level netlist describes synthesized logic gates and connections. Layout describes the physical placement and routing of those gates on silicon. This work focuses only on RTL.

Q: What is a Trojan trigger?
Answer: A trigger is the condition that activates a Trojan. It may depend on a rare input sequence, a counter value, an internal state, or a combination of signals.

Q: What is a Trojan payload?
Answer: A payload is the malicious action that occurs after the trigger activates. It may modify data, alter control logic, leak a secret, or disrupt normal operation.

Q: What is the difference between a bug and a Trojan?
Answer: A bug is generally unintentional. A Trojan is intentionally inserted malicious logic. Both may create incorrect behavior, but their security implications and threat models are different.

Q: Why might a Trojan escape normal simulation?
Answer: A Trojan can use a rare trigger condition that standard test benches do not activate. Simulation also cannot exhaustively test all possible states in a complex design.

Q: Can RTL analysis detect every Hardware Trojan?
Answer: No. RTL analysis cannot cover all Trojans, especially analog, gate-level, doping-level, or physical-layout attacks. It is one layer of a broader hardware-security workflow.

Q: What are conventional methods for Hardware Trojan detection?
Answer: Common approaches include functional testing, formal verification, side-channel analysis, structural analysis, machine learning, and deep learning. Each method has different requirements and limitations.

Q: Why not rely only on formal verification?
Answer: Formal verification can provide strong guarantees for specified properties, but it may be computationally expensive and requires a suitable property specification. The proposed method can help screen suspicious RTL regions for further formal analysis.

2. Motivation and Research Contribution

Q: What problem does this research address?
Answer: This work studies RTL Hardware Trojan detection and localization using an LLM while keeping adaptation practical under limited computing resources.

Q: What is the main objective of the work?
Answer: The objective is to build a resource-efficient framework that classifies RTL as clean or Trojan-inserted and identifies suspicious trigger and payload regions.

Q: What is the central contribution?
Answer: The central contribution is an end-to-end controlled framework combining RTL preprocessing, synthetic Trojan generation, prompt sanitization, LoRA-based adaptation, detection, and localization-aware evaluation.

Q: What is novel in this work?
Answer: The novelty is the integrated engineering workflow for controlled RTL Trojan analysis using lightweight LoRA adaptation and localization metadata. The work does not claim a new transformer architecture or a new LoRA algorithm.

Q: Why use an LLM for RTL analysis?
Answer: RTL is structured code containing declarations, conditions, assignments, and dependencies. Transformer-based LLMs can learn contextual relationships from serialized code-like representations.

Q: Does an LLM truly understand hardware semantics?
Answer: Not in the same way as a hardware engineer or a formal tool. It learns statistical and structural patterns from data. Strong results indicate useful task performance in the tested setting, not complete semantic understanding.

Q: Why is resource efficiency important?
Answer: Full LLM fine-tuning can require large GPUs and substantial memory. Resource efficiency makes experiments more accessible and reproducible for academic researchers.

Q: How is your work different from a normal code-classification task?
Answer: The task involves hardware-security reasoning over RTL, controlled Trojan trigger-payload insertion, and localization of suspicious RTL regions. The security interpretation and localization ground truth make it different from ordinary code classification.

3. Dataset Engineering

Q: Where did the clean RTL samples come from?
Answer: The clean Verilog RTL samples were obtained from the RTL-Coder repository. These samples form the base set for controlled Trojan generation.

Q: How large is the final dataset?
Answer: The final dataset contains 600 RTL samples: 300 clean samples and 300 Trojan-inserted samples.

Q: Why is the dataset balanced?
Answer: A balanced dataset prevents a classifier from gaining high accuracy merely by predicting the majority class. Here, clean and Trojan classes contain equal numbers of samples.

Q: How were Trojan-inserted samples created?
Answer: Controlled trigger and payload logic were inserted into clean Verilog RTL modules. The insertion process also recorded the affected trigger and payload regions as metadata.

Q: Why use synthetic Trojan generation?
Answer: Synthetic generation provides known ground truth for both classification and localization. It also makes the experiment reproducible, although it limits real-world generalization.

Q: Does the dataset contain real industrial Trojans?

Answer: No. The current dataset uses controlled synthetic RTL Trojan insertions. Real-world and heterogeneous benchmark evaluation remains future work.

Q: How is the dataset split?
Answer: The split is randomized and stratified: 420 samples for training, 90 for validation, and 90 for testing. Each split has equal clean and Trojan class counts.

Q: How many clean and Trojan samples are in the test set?
Answer: The test set contains 45 clean samples and 45 Trojan-inserted samples.

Q: Why use a randomized stratified split?
Answer: Randomization supports a conventional held-out evaluation, while stratification preserves class balance across training, validation, and test sets.

Q: Does randomized splitting guarantee generalization?
Answer: No. Similar RTL structures or insertion patterns may still appear across splits. The paper explicitly states that template-level separation was not enforced.

Q: Could related RTL templates appear in both training and testing data?
Answer: Yes, that cannot be ruled out. This may make the reported performance optimistic relative to a template-separated or cross-project evaluation.

Q: How should future work improve the split?
Answer: Future work should use design-family-level, template-level, or project-level splits. This would better test performance on structurally novel RTL designs.

Q: How did you preserve valid RTL functionality after Trojan insertion?
Answer: The procedure used controlled and syntactically valid insertion patterns intended to preserve valid RTL behavior. However, the paper does not claim exhaustive functional equivalence checking, so future work should add compilation, simulation, and formal validation for every generated sample.

4. Trojan Injection and Prompt Sanitization

Q: What is the purpose of recording `trigger_nodes` and `payload_nodes`?
Answer: They provide ground-truth localization metadata. During evaluation, predicted suspicious regions are compared against these recorded trigger and payload regions.

Q: What are `HT_START` and `HT_END` markers?
Answer: They are temporary generation-time markers used to identify inserted Trojan regions. They are not part of the final model prompt.

Q: Why remove explicit Trojan markers before training and inference?
Answer: If markers or Trojan-specific comments remained, the model could classify samples using obvious textual labels instead of RTL structure. Removing them reduces explicit shortcut learning.

Q: What is prompt sanitization?
Answer: Prompt sanitization removes temporary markers, Trojan-specific comments, and explicit annotations while retaining the RTL structural content needed for analysis.

Q: Does removing markers eliminate all data leakage?
Answer: No. It removes explicit label leakage, but synthetic insertion templates, coding patterns, or similar RTL structures may still create indirect leakage.

Q: Can you call the evaluation completely leakage-free?
Answer: The safer term is *label-sanitized evaluation*. Explicit annotations were removed, but the randomized split does not fully exclude structural similarity across partitions.

Q: Are localization labels shown to the model during inference?
Answer: No. The ground-truth trigger and payload metadata are retained separately for evaluation. During inference, the model receives the RTL prompt without the ground-truth localization annotations.

Q: Are classification labels shown to the model during training?
Answer: The model requires task supervision during adaptation, but the key leakage-control point is that the input RTL prompt does not contain direct Trojan markers or embedded ground-truth annotations. At inference, labels are hidden.

Q: What is structural obfuscation in this context?
Answer: It refers to lightweight transformations intended to reduce reliance on superficial naming or formatting patterns while preserving RTL structural relationships.

Q: Can preprocessing itself introduce artifacts?
Answer: Yes. Any deterministic processing pipeline can create regularities. Robust ablations and external evaluation are needed to verify that preprocessing improves rather than biases generalization.

5. LLM and Transformer Concepts

Q: What is a large language model?
Answer: A large language model is a neural network trained to model token sequences. It learns statistical relationships between tokens and can be adapted for tasks such as classification, generation, code analysis, and question answering.

Q: What is a transformer?
Answer: A transformer is a neural-network architecture that uses attention mechanisms to model relationships between tokens in a sequence. It is the main architecture behind modern LLMs.

Q: What is self-attention?
Answer: Self-attention allows each token to assign importance to other tokens in the sequence. In RTL, this can help the model connect signals, conditions, assignments, and declarations that may be far apart in the text.

Q: Why is attention useful for RTL?
Answer: RTL contains long-range dependencies. For example, a signal may be declared in one place, assigned elsewhere, and used in a conditional block later. Attention can model these relationships.

Q: What is tokenization?
Answer: Tokenization converts text into smaller units called tokens that the model can process. Verilog symbols, keywords, identifiers, operators, and numbers are represented as token sequences.

Q: Can tokenization create challenges for RTL?
Answer: Yes. Long identifiers, code formatting, uncommon syntax, and long modules can increase token length or fragment meaningful RTL constructs. This is another reason why code-aware preprocessing matters.

Q: What is Qwen2.5-1.5B-Instruct?
Answer: It is an instruction-tuned LLM with approximately 1.5 billion parameters. It was selected because it provides practical model capacity with manageable resource requirements.

Q: Why choose a 1.5-billion-parameter model?
Answer: It provides a balance between capability and accessibility. Much larger models may be more costly to adapt, while much smaller models may have less capacity for complex RTL pattern learning.

Q: Why use an instruction-tuned model?
Answer: Instruction-tuned models are designed to follow task-oriented prompts. This aligns with the instruction-oriented RTL prompt format used in the workflow.

Q: Could a graph neural network be better for this task?
Answer: Possibly. Hardware designs naturally form graphs of signals and logic dependencies. A graph neural network is an important baseline, but fair comparison requires the same dataset, split, labels, and evaluation protocol.

6. LoRA, PEFT, and Training

Q: What is fine-tuning?
Answer: Fine-tuning adapts a pretrained model to a domain-specific task using task-specific data. In this work, the target task is RTL Trojan detection and localization.

Q: What is PEFT?
Answer: PEFT means Parameter-Efficient Fine-Tuning. It adapts a model while updating only a small fraction of parameters.

Q: What is LoRA?
Answer: LoRA stands for Low-Rank Adaptation. It adds small trainable low-rank matrices to selected model weights while keeping most pretrained model parameters frozen.

Q: What is the mathematical idea behind LoRA?
Answer: Instead of fully learning a large update ΔW, LoRA approximates it using:

$$\Delta W = BA,$$

where A and B are low-rank trainable matrices. The adapted weight becomes:

$$W' = W + BA.$$

The original pretrained weight W remains frozen.

Q: Why does LoRA save memory?
Answer: The optimizer, gradients, and updates are stored mainly for small adapter matrices rather than for the full backbone. This substantially reduces the trainable state compared with full fine-tuning.

Q: What is the difference between LoRA and full fine-tuning?
Answer: Full fine-tuning updates most or all model parameters. LoRA updates only small adapter parameters while preserving the base model weights.

Q: Does LoRA always perform as well as full fine-tuning?
Answer: No. LoRA trades adaptation capacity for efficiency. It can be highly effective, but full fine-tuning may perform better for some tasks at much higher computational cost.

Q: What was the LoRA rank in your work?
Answer: The reported LoRA rank was 8. It controls the capacity of the low-rank adapter update.

Q: Why choose rank 8?
Answer: Rank 8 was used as the selected low-resource configuration. A more extensive study should compare multiple ranks to quantify the performance-resource trade-off.

Q: What is QLoRA?
Answer: QLoRA combines LoRA adapters with low-bit quantization of the base model. It can reduce memory use further, but the final reported workflow used FP16 LoRA adaptation for stability and reproducibility.

Q: What is FP16?
Answer: FP16 is 16-bit floating-point precision. It reduces memory use and can improve training speed compared with FP32, subject to numerical-stability considerations.

Q: What was the learning rate?
Answer: The learning rate was 2×10^{-4}, also written as `2e-4`.

Q: What was the batch size?
Answer: The physical batch size was 1. This keeps GPU memory usage low.

Q: What is gradient accumulation?
Answer: Gradient accumulation adds gradients over several steps before updating the parameters. The reported gradient accumulation value was 8.

Q: Why use batch size 1 with gradient accumulation 8?
Answer: This configuration allows training within a limited-memory GPU environment while approximating an effective larger batch for parameter updates.

Q: How many epochs were used?
Answer: The training configuration used 3 epochs.

Q: How many parameters were trainable?
Answer: The LoRA adapters contained 2.17 million trainable parameters, reported as 0.141% of the total model parameters.

Q: Why is only 0.141% trainable?
Answer: Most Qwen backbone parameters remain frozen. Only lightweight LoRA adapter parameters are optimized for the task.

Q: Which exact model layers did LoRA target?
Answer: The accepted paper reports the LoRA rank and trainable parameter count but does not provide a complete target-layer specification. I would not claim details that are not documented in the paper.

Q: What hardware was used?
Answer: The implementation used Google Colab with an NVIDIA Tesla T4 GPU.

Q: Is Tesla T4 a consumer GPU?
Answer: Tesla T4 is a data-center GPU. The practical point is that it is commonly accessible in academic cloud environments such as Google Colab, without requiring dedicated multi-GPU infrastructure.

Q: What does the approximately six-minute runtime mean?
Answer: It is the reported approximate training runtime for the controlled setup. Exact time can vary with cloud allocation, software versions, initialization, data loading, and hardware availability.

7. Detection and Localization Metrics

Q: What binary classification metrics are used?
Answer: The work reports accuracy, precision, recall, and F1-score.

Q: What is accuracy?
Answer: Accuracy is the fraction of all correctly classified samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Q: What is precision?
Answer: Precision measures the reliability of positive Trojan predictions:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Q: What is recall?
Answer: Recall measures the fraction of actual Trojan samples that are detected:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Q: What is F1-score?
Answer: F1-score is the harmonic mean of precision and recall:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Q: What does a false positive mean in your task?
Answer: A false positive means that a clean RTL sample is incorrectly classified as Trojan-inserted.

Q: What does a false negative mean in your task?
Answer: A false negative means that a Trojan-inserted RTL sample is incorrectly classified as clean. In a hardware-security context, this can be particularly serious.

Q: What is TLC?
Answer: TLC means Trigger Localization Coverage. It measures how much of the ground-truth trigger region is covered by the predicted trigger region:

$$\mathrm{TLC} = \frac{|T_p \cap T_g|}{|T_g|}.$$

Q: What is PLC?
Answer: PLC means Payload Localization Coverage. It measures how much of the ground-truth payload region is covered by the predicted payload region:

$$\mathrm{PLC} = \frac{|P_p \cap P_g|}{|P_g|}.$$

Q: What do T_p and T_g represent?
Answer: T_p is the predicted trigger-node set, while T_g is the ground-truth trigger-node set recorded during controlled Trojan insertion.

Q: What do P_p and P_g represent?
Answer: P_p is the predicted payload-node set, while P_g is the recorded ground-truth payload-node set.

Q: Do TLC and PLC measure localization precision?
Answer: No. They are coverage-oriented metrics. They measure recovery of ground-truth nodes, but they do not fully penalize predicting extra irrelevant nodes.

Q: What additional localization metrics would strengthen the work?
Answer: Localization precision, exact-match rate, region-level F1-score, and intersection-over-union would provide a more complete evaluation.

8. Experimental Results

Q: What was the test-set accuracy?
Answer: The model achieved 98.89% accuracy on the 90-sample controlled test set.

Q: What was the test-set precision?
Answer: Precision was 100.00% for Trojan predictions.

Q: What was the test-set recall?
Answer: Recall was 97.78%.

Q: What was the F1-score?
Answer: The F1-score was 98.88%.

Q: What was the confusion matrix?
Answer: The test set contained 45 clean and 45 Trojan samples. The model correctly classified 45 clean samples and 44 Trojan samples. It produced zero false positives and one false negative.

Q: Why is accuracy 98.89%?
Answer: There were 89 correct predictions out of 90 test samples:

$$\frac{89}{90} \times 100 = 98.89\%.$$

Q: Why is precision 100%?
Answer: There were 44 true Trojan predictions and zero false-positive Trojan predictions:

$$\frac{44}{44+0} = 100\%.$$

Q: Why is recall 97.78%?
Answer: There were 44 detected Trojan samples and one missed Trojan sample:

$$\frac{44}{44+1} = 97.78\%.$$

Q: What were the localization results?
Answer: The reported Trigger Localization Coverage was 98.89%, and the reported Payload Localization Coverage was also 98.89%.

Q: Can you claim that the model is 98.89% accurate for all Hardware Trojans?
Answer: No. The correct statement is that the model achieved 98.89% accuracy on the controlled randomized test set used in this study.

Q: Why report both detection and localization?
Answer: Binary detection identifies whether a sample appears suspicious. Localization adds practical value by identifying the trigger and payload regions that may require engineering review.

9. Comparison, Limitations, and Threat Model

Q: How does this work compare with PEARL?
Answer: PEARL is important related work in LLM-based Hardware Trojan analysis. However, it uses a different model, adaptation approach, evaluation setting, and hardware environment, so it is treated as contextual prior work rather than a direct benchmark.

Q: Does your work outperform PEARL?
Answer: I do not claim that. The experiments are not directly matched, and comparing raw performance numbers across different datasets and protocols would be scientifically misleading.

Q: Why did you not compare with zero-shot prompting?
Answer: A zero-shot baseline would be a useful ablation. The accepted paper focuses on the controlled LoRA-based workflow and does not report that baseline.

Q: Why did you not compare with classical machine learning or graph neural networks?
Answer: Such comparisons would strengthen the evaluation. They should use identical splits, preprocessing, and metrics to make the comparison meaningful.

Q: What is the main limitation of the dataset?
Answer: The data are controlled synthetic Trojan samples. Synthetic patterns may not capture the diversity and stealth of real-world Hardware Trojans.

Q: What is the main limitation of the split?
Answer: The randomized stratified split does not enforce template-level separation. Therefore, structural similarity between training and test data cannot be ruled out.

Q: What is the main limitation of the localization metrics?
Answer: TLC and PLC measure coverage but do not fully measure false localization. Additional precision-oriented localization metrics are needed.

Q: What is the main limitation of the hardware scope?
Answer: The work analyzes RTL only. It does not evaluate gate-level netlists, post-synthesis representations, physical layouts, analog Trojans, or manufacturing-level attacks.

Q: Can an adaptive attacker evade this method?
Answer: Potentially, yes. An attacker may use different Trojan structures, uncommon RTL patterns, adversarial transformations, or post-synthesis modifications. Robustness against adaptive attackers needs dedicated testing.

Q: Can this work prove a design is Trojan-free?
Answer: No. This is a learned screening and localization tool, not a formal proof of hardware security. Formal verification and expert review remain necessary.

Q: Can the LLM hallucinate a suspicious location?
Answer: Yes. LLM predictions can be incorrect or overconfident. Localization should guide review and downstream verification rather than serve as an autonomous security verdict.

Q: Is this production-ready for industry?
Answer: No. It is a controlled engineering prototype. Production use would require real-world benchmarks, stronger baselines, grouped splits, formal validation, robustness tests, and integration with established verification flows.

10. Tough Professor Questions and Best Defence

Q: Are you detecting Trojans or merely detecting your own generated pattern?
Answer: The current experiment demonstrates detection and localization of controlled Trojan-inserted RTL after removal of explicit markers. It does not establish performance against every attacker-designed Trojan. That is why the paper presents the work as a reproducible prototype and identifies broader benchmark evaluation as future work.

Q: Your high result could be due to template leakage. Why should we accept it?
Answer: That concern is valid. We reduce explicit leakage through sanitization, but the randomized split cannot fully eliminate structural similarity. The result is still useful as controlled evidence for the proposed workflow, while template-separated evaluation is needed before making stronger generalization claims.

Q: Why should this be considered research rather than just fine-tuning a model?
Answer: The contribution is not merely applying LoRA. It is the design and evaluation of a complete controlled RTL security-analysis workflow that includes Trojan generation, localization metadata, prompt sanitization, lightweight adaptation, and explicit limitations.

Q: What would be the most important ablation study?
Answer: The most important ablation would compare zero-shot prompting, LoRA adaptation, full fine-tuning, and non-LLM baselines under the same template-separated dataset split.

Q: What would be the most important next experiment?
Answer: The most important next experiment is evaluation on larger, heterogeneous, template-separated, and ideally real-world RTL Trojan benchmarks.

Q: What would happen if you increased LoRA rank?
Answer: A higher rank would increase adapter capacity and trainable parameters. It might improve performance or might overfit the small synthetic dataset. The accuracy-resource trade-off should be measured empirically.

Q: What would happen if you used a larger LLM?
Answer: A larger model may have more capacity, but it would increase memory, training time, and cost. The key question is whether the performance improvement justifies the additional resource requirement.

Q: What would happen if you used full fine-tuning?
Answer: Full fine-tuning may provide more adaptation capacity but requires substantially more memory and computation. A direct comparison would quantify whether LoRA gives a favorable performance-efficiency trade-off.

Q: How would you validate functional correctness of generated Trojan RTL?
Answer: I would add automated compilation, simulation with regression test benches, linting, and formal

equivalence or property checking. This would strengthen confidence that each generated sample is valid and that the intended design behavior is controlled.

Q: If you do not know an implementation detail during the presentation, what should you say?
Answer: I would say: "That exact detail is not reported in the accepted paper, so I do not want to make an unsupported claim. The appropriate way to establish it would be through a documented ablation or reproducibility experiment."

11. Final Statements to Memorize

One-sentence contribution:
"We present a controlled, LoRA-based, resource-efficient framework for RTL Hardware Trojan detection and trigger-payload localization using Qwen2.5-1.5B-Instruct."

One-sentence result:
"On our controlled balanced 90-sample test set, the framework achieved 98.89% accuracy, 98.88% F1-score, and 98.89% trigger and payload localization coverage."

One-sentence limitation:
"The dataset is synthetic and the randomized split does not enforce template-level separation, so the results should not yet be generalized to all real-world Hardware Trojans."

One-sentence future work:
"The next step is template-separated evaluation on larger and more diverse real-world RTL benchmarks, supported by matched baselines and stronger localization metrics."

One-sentence safety claim:
"This method is an intelligent screening and localization aid; it complements formal verification and expert review rather than replacing them."